

Giới thiệu Tối ưu hóa (Optimization)

Tối ưu hoá – AI203DV01
(Optimization)
Vũ Đình Khôi

Nội dung

- Giới thiệu (Introduction and Background)
- Tối ưu một chiều (One-Dimensional Optimization)
- Tối ưu đa chiều (Multidimensional Optimization)
- Case Study
 - Equilibrium and Minimum Potential Energy

- Chapter 7. Steven Chapra, David Clough. (2022). *Applied Numerical Methods with Python for Engineers and Scientists*. McGraw Hill
- Chapter 2. Serge Kruk. (2018). *Practical Python AI Projects: Mathematical Models of Optimization Problems with Google OR-Tools*. Apress

**I find that I don't understand things unless I try
to program them.**



Donald E. Knuth

Professor Emeritus at Stanford University

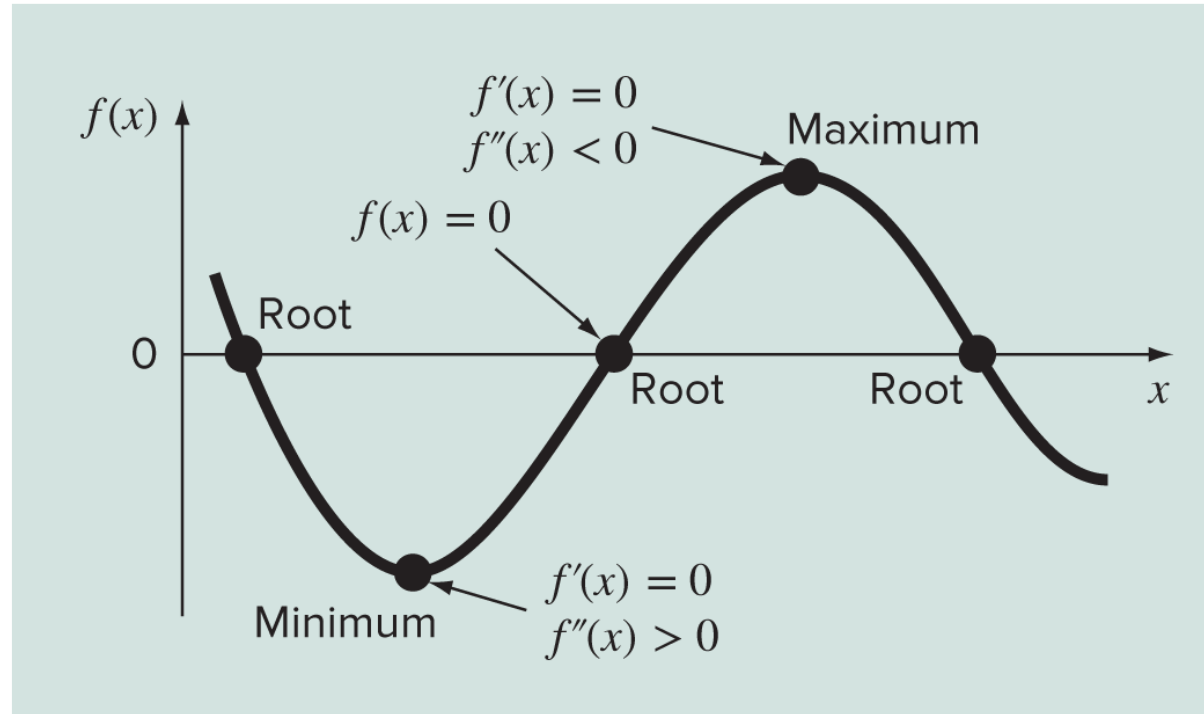
If you want to **master something**, teach it



-- **Richard Feynman**

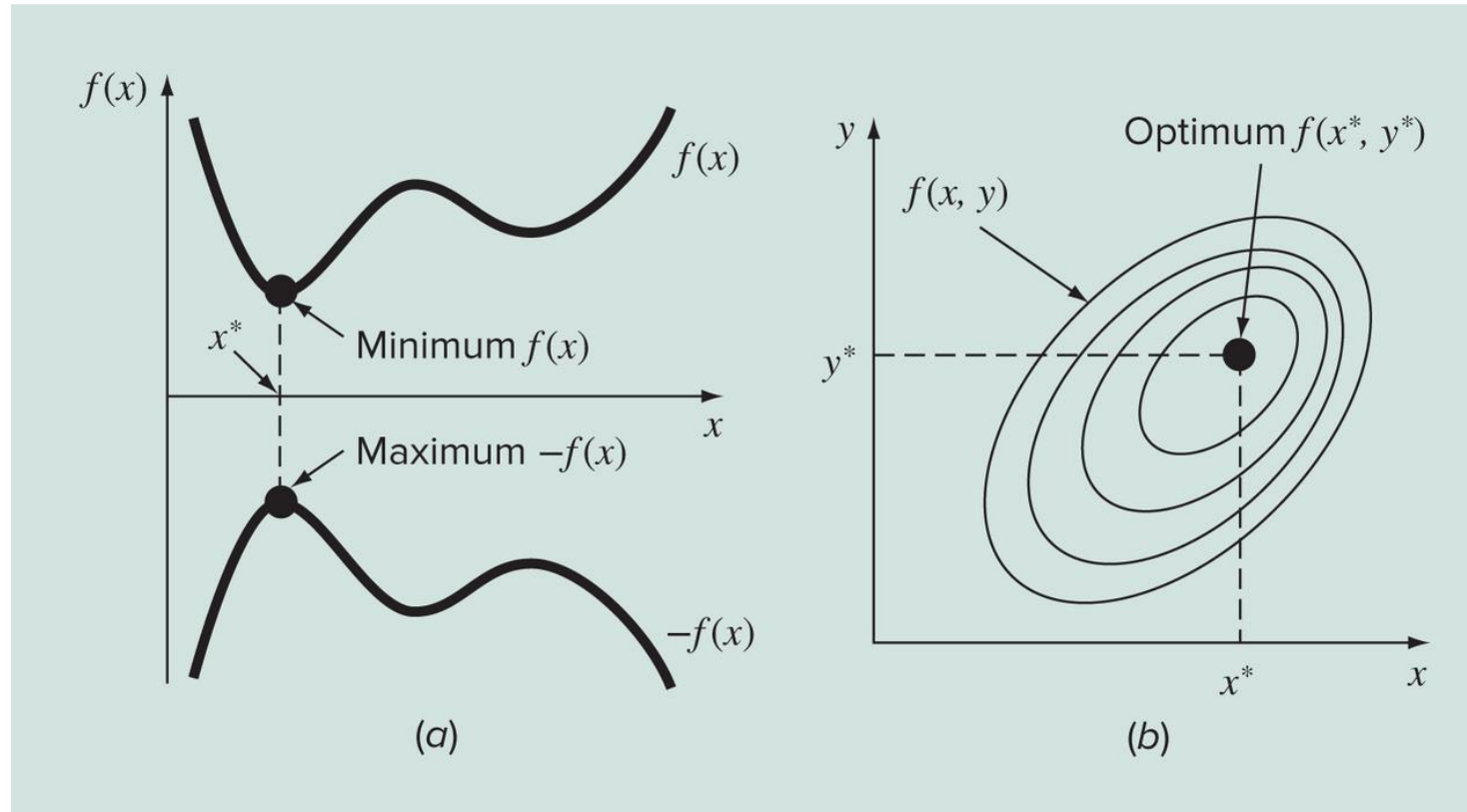
- Bài toán tối ưu (optimization problem) là bất kỳ bài toán nào có dạng/ được trình bày dưới dạng (formulate)?
 - Trả lời câu hỏi “Cái gì tốt nhất cho...?” (“What is the best ... ?”)
 - Tốt nhất (the best) là gì?
 - Nhanh nhất, ngắn nhất, hài lòng nhất, ít nhất... (fastest, shortest, most pleasant to ride, least bumpy, or least fuel-guzzling?)
- Ví dụ
 - What is the best route to get from home to work?
 - What is the best way to produce cars to maximize profit?
 - What is the best way to carry groceries home: paper or plastic?
 - Which is the best school for my kid?
 - Which is the best fuel to use in rocket boosters?
 - What is the best placement of transistors on a chip?
 - What is the best NBA schedule?

- Khía cạnh toán học, tối ưu hóa liên quan đến tìm cực đại/cực tiểu của hàm một biến hoặc nhiều biến.



Giới thiệu

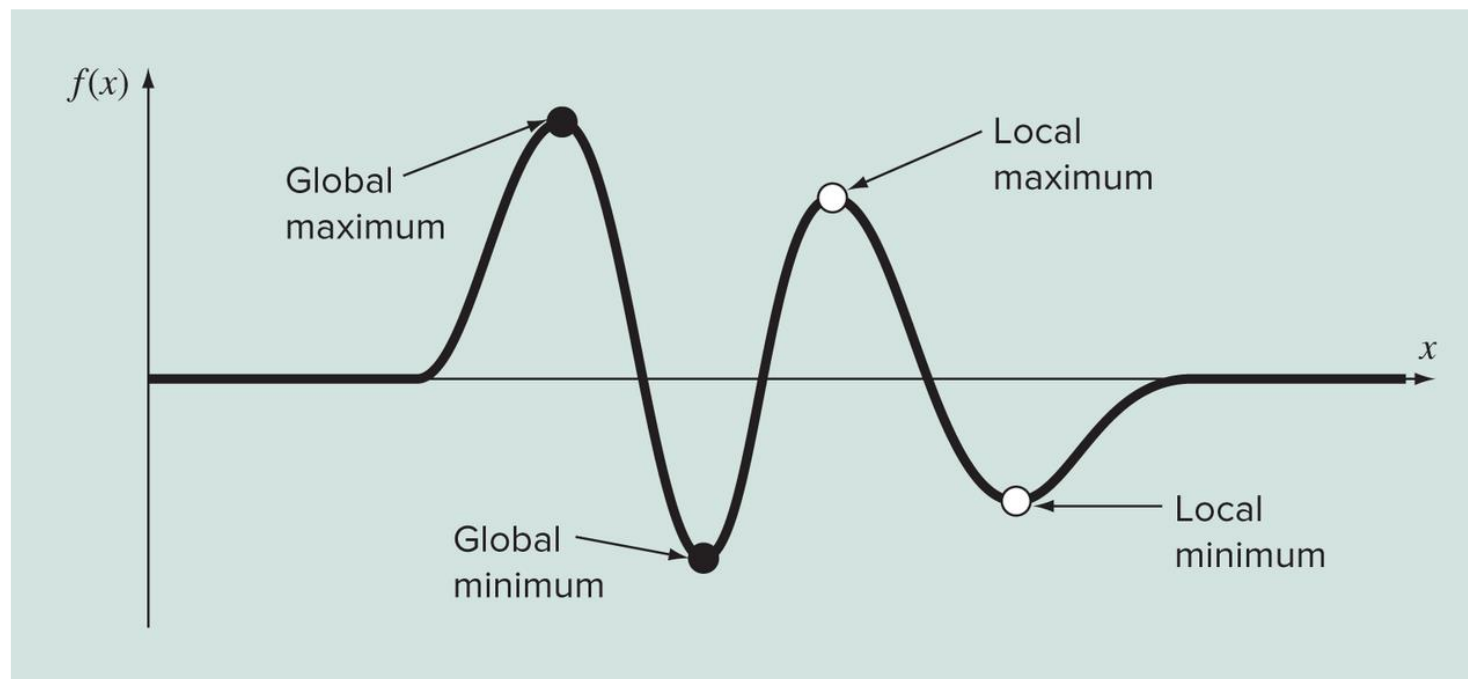
- Cực tiểu của hàm $f(x)$ tương đương cực đại của hàm $-f(x)$
- Tối ưu trên không gian một chiều hoặc nhiều chiều



Tối ưu hóa trên không gian một chiều

■ One-dimensional Optimization

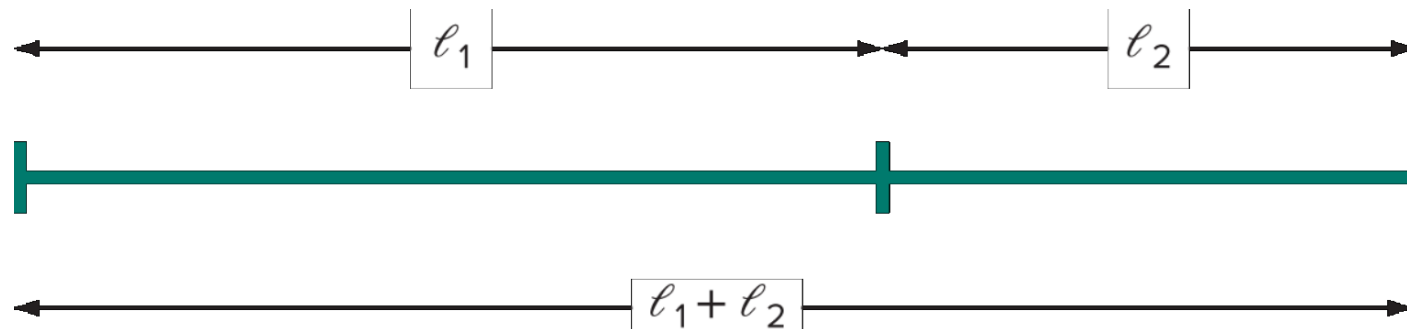
- Cực trị (extremum) toàn cục (global) hoặc cục bộ (local)
- Tối ưu toàn cục (global optima) hoặc cục bộ (local optima)



Tìm kiếm tỷ lệ vàng (Golden-section method)

■ Theo Euclid, tỷ lệ vàng (golden ratio) ϕ (phi)

- ϕ là số ma thuật (magical number), để ý $1/\phi = \phi - 1$



$$\frac{l_1 + l_2}{l_1} = \frac{l_1}{l_2} = \phi \rightarrow 1 + \frac{l_2}{l_1} = \phi \rightarrow 1 + \frac{1}{\phi} = \phi \rightarrow \phi^2 - \phi - 1 = 0$$

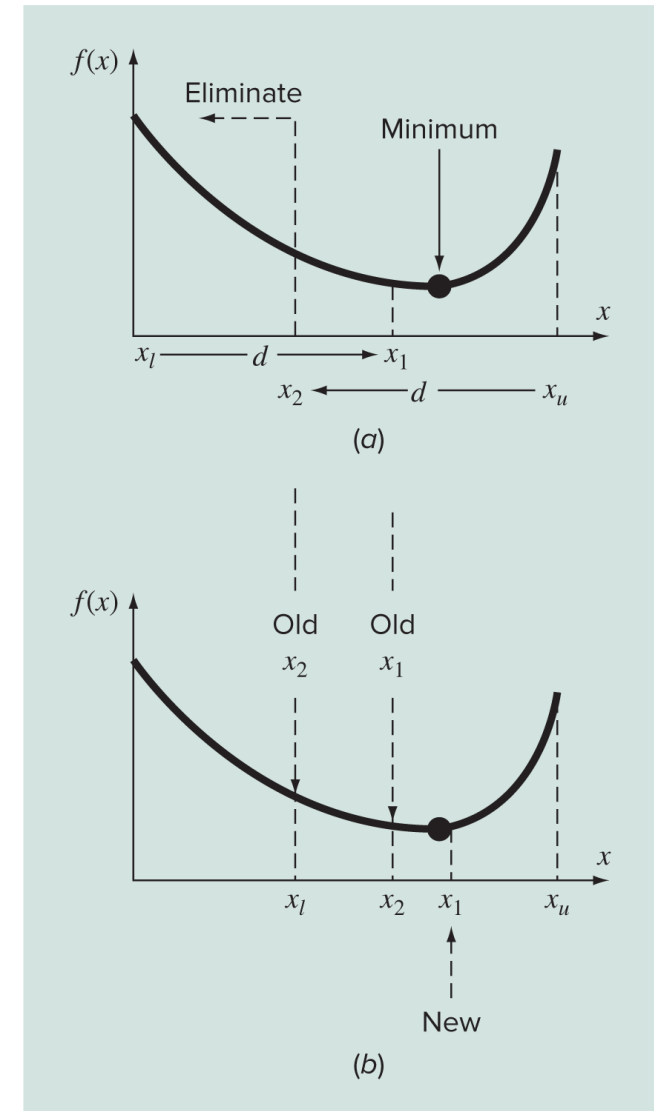
$$\phi = \frac{1 + \sqrt{5}}{2} \cong 1.61803 (\phi > 0) \quad \text{Để ý } \frac{1}{\phi} = \phi - 1$$

Tìm kiếm tỷ lệ vàng (Golden-section method)

- Giả sử điểm cực trị cần tìm nằm trong được giới hạn (bracket) trong khoảng $[x_l, x_u]$ của một hàm đơn điệu (unimodal function)
 - x_l là cận dưới (lower) và x_u là cận trên (upper)

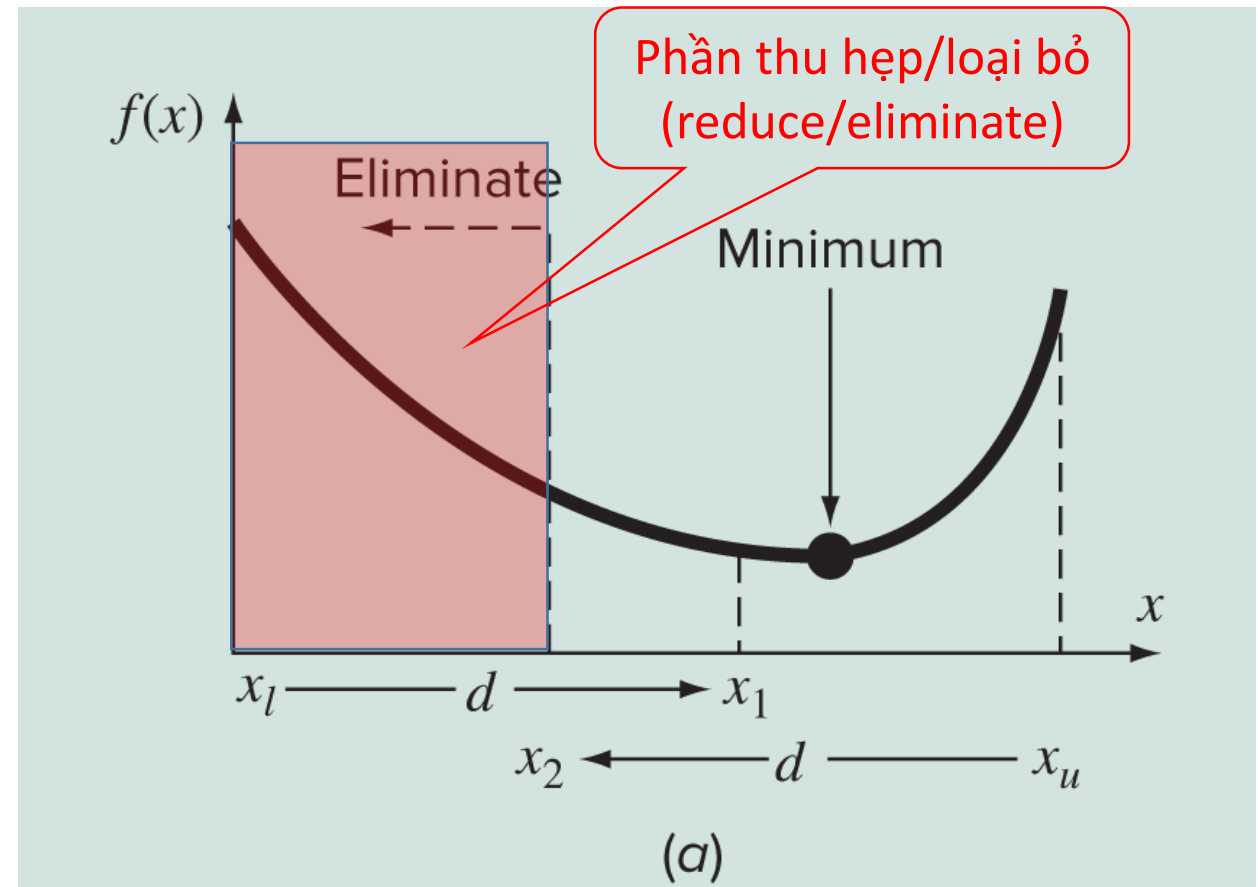
Khác với bài toán tìm nghiệm chỉ cần tính 2 giá trị $f(x)$ và xác định cùng dấu/trái dấu để thu hẹp khoảng chứa nghiệm. Trong bài toán tìm cực trị cần 3 giá trị $f(x)$

- Lưu ý x_2 cũ thành x_1 mới và x_1 cũ thành x_2 mới. Do đó không cần tính lại các giá trị $f(x)$ cho lần lặp kế tiếp.



Tìm kiếm tỷ lệ vàng (Golden-section method)

- Ý tưởng cơ bản là thu hẹp khoảng chứa cực trị bên phải/trái theo một tỷ lệ nào đó?
- Với $d = (\phi - 1)(x_u - x_l)$
- Nếu $f(x_1) < f(x_2)$: cực tiểu nằm trong khoảng (x_2, x_u)
- Nếu $f(x_1) > f(x_2)$: cực tiểu nằm trong khoảng (x_l, x_1)



Tìm kiếm tỷ lệ vàng (Golden-section method)



$$d = (\phi - 1)(x_u - x_l) \quad \text{và} \quad \frac{1}{\phi} = \phi - 1 \Rightarrow d = \frac{1}{\phi}(x_u - x_l)$$

$$x_1 = x_l + d$$

$$\text{hay } (x_u - x_l) = \phi d$$

$$x_{2\text{new}} = x_u - d' = x_u - (\phi - 1)(x_u - x_{l\text{new}})$$

$$\text{mà } x_{l\text{new}} = x_u - d$$

$$\Rightarrow x_{2\text{new}} = x_u - (\phi - 1)(x_u - x_u + d)$$

$$\begin{aligned} \text{Vậy } (x_{2\text{new}} - x_1) &= x_u - (\phi - 1) \cdot d = x_u - \phi d + d \\ &= x_u - \phi d + d - x_l - d \end{aligned}$$

$$= (x_u - x_l) - \phi d = \phi d - \phi d = 0$$

$$\Rightarrow \boxed{x_{2\text{new}} = x_1}$$

Tìm kiếm tỷ lệ vàng (Golden-section method)

- Đọc thêm slide *Numerical Analysis_11_Tối ưu hóa.pdf*

scipy.optimize.minimize_scalar

- Tìm giá trị cực tiểu của hàm số $f(x)$
 - $f(x)$: hàm vô hướng một biến (a single variable scalar function)
- Sử dụng các phương pháp/thuật toán
 - Brent (mặc định)
 - Golden-Section Search
 - ...

- Ví dụ

```
from scipy.optimize import minimize_scalar
```

```
def func(x):  
    return x**2 + 2*x + 1
```

```
result = minimize_scalar(func)  
print(result)
```

scipy.optimize.minimize

- Tìm giá trị cực tiểu của hàm số $f(x, y)$
 - $f(x, y)$: hàm vô hướng nhiều biến (a multivariate scalar function)
- Sử dụng các phương pháp/thuật toán
 - BFGS
 - Nelder-Mead
 - CG...
- Ví dụ

```
from scipy.optimize import minimize
```

```
def func(x):  
    return x[0]**2 + x[1]**2
```

```
result = minimize(func, [0, 0])  
print(result)
```

Linear programming & Integer programming

- Quy hoạch tuyến tính (LP)
- Quy hoạch nguyên (IP)

Google OR-Tools

- Phần mềm nguồn mở cho các bài toán tối ưu hóa kết hợp (combinatorial optimization)
 - OR: Operation Research (Vận trù học)
 - <https://github.com/google/or-tools>
 - <https://developers.google.com/optimization>
- Hỗ trợ các ngôn ngữ
 - C++, Python, Java, .Net
- Lịch sử

- Một số link ứng dụng thực tế

- Video Stabilization on YouTube

<https://research.google/blog/video-stabilization-on-youtube/>

https://www.youtube.com/watch?v=Pr_fpbAok8

- Sudoku, Linear Optimization, and the Ten Cent Diet

<https://research.google/blog/sudoku-linear-optimization-and-the-ten-cent-diet/>

- Một số bài toán trình giải (OR-Tools solvers) có thể giải
 - Định tuyến/Lộ trình phương tiện (Vehicle routing): tìm các lộ trình tối ưu cho việc giao nhận hàng cùng với các ràng buộc cho trước.
 - Lập lịch (Scheduling): lập lịch tối ưu cho tập các công việc phức tạp với ràng buộc trình tự, nguồn lực tài nguyên...
 - Đóng hàng (Bin packing): đóng càng nhiều càng tốt hàng hóa có kích thước khác nhau vào số lượng các thùng chứa cố định với dung lượng tối đa.
- Các bài toán tối ưu thường có rất nhiều nghiệm (vast number of possible solutions)
 - OR-Tools sử dụng các thuật toán tiên tiến nhất hiện nay (state-of-the-art algorithms/SOTA algorithms) để thu hẹp tập hợp tìm kiếm và trả về nghiệm tối ưu (hoặc gần như tối ưu).

Google OR-Tools

- OR-Tools gồm các trình giải (solvers) cho các lớp bài toán
 - Quy hoạch ràng buộc (Constraint Programming)
 - Quy hoạch nguyên-hỗn hợp (Linear and Mixed-Integer Programming)
 - GLOP (Google Linear Optimization Package)
 - SCIP (Solving Constraint Integer Programs)
 - Định tuyến phương tiện (Vehicle Routing)
 - Đồ thị (Graph Algorithms)

CP-SAT



Constraint Programming with CP-SAT

Using the CP-SAT Solver

Solving a CP Problem

Cryptarithmic

The N-queens Problem

Setting Solver Limits

Packing



Overview

The Knapsack Problem

Multiple Knapsacks

Bin Packing Problem

Routing



Overview

Traveling Salesperson Problem

Vehicle Routing Problem

Capacity Constraints

Pickups and Deliveries

Time Window Constraints

Dimensions

Resource Constraints

Penalties and Dropping Visits

Common Routing Tasks

Routing Options

Google Directions API

Original CP Solver

Linear Optimization



- Overview
- MPSolver Interface
- Solving an LP Problem
- Advanced LP Solving
- The Stigler Diet Problem
- Service for Apps Script 
- Mathematical background for PDLP

Scheduling

- Overview
- Employee Scheduling
- The Job Shop Problem



Integer Optimization



- Overview
- Solving a MIP Problem
- Using Arrays to Define a Model

Network Flows



- Overview
- Maximum Flows
- Minimum Cost Flows
- Assignment as a Min Cost Flow Problem

Assignment



- Overview
- Solving an Assignment Problem
- Assignment with Teams of Workers
- Assignment with Task Sizes
- Assignment with Allowed Groups
- Linear Sum Assignment Solver

Một số ký pháp (notation)

Table 1-1. *Equivalence of Expression in Math and Python Modes*

Object	Math Mode	Python Mode
Scalar Variable	X	<code>X</code>
Vector	v_i	<code>v[i]</code>
Matrix	M_{ij}	<code>M[i][j]</code>
Inequality	$x + y \leq 10$	<code>x+y <= 10</code>
Summation	$\sum_{i=0}^9 x_i$	<code>sum(x[i] for i in range(10))</code>
Set Definition	$\{i^2 \mid i \in [0, 1, \dots, 9]\}$	<code>[i**2 for i in range(10)]</code>

Google OR-Tools: example 1

- Tìm giá trị cực đại của $3x + y$ thỏa các ràng buộc sau
(Maximize $3x + y$ subject to the following constraints):
 - $0 \leq x \leq 1$
 - $0 \leq y \leq 2$
 - $x + y \leq 2$
- Biến quyết định (decision variables)
 - ...
- Các ràng buộc (constraints)
 - ...
- Hàm mục tiêu (objective function)
 - ...

Google OR-Tools: example 1

```
9 from ortools.linear_solver import pywraplp
10
11
12 def solve_model():
13     # Tạo linear solver với GLOP (Google Linear Optimization)
14     # solver = pywraplp.Solver.CreateSolver('GLOP')
15     solver = pywraplp.Solver("Simple Linear Programming Example",
16                             pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
17
18     # Các biến quyết định (Decision variables)
19     x = solver.NumVar(0, 1, "x")
20     y = solver.NumVar(0, 2, "y")
21
22     # Các ràng buộc (Constraints)
23     solver.Add(x + y <= 2)
24
```

Google OR-Tools: example 1

```
25     # Hàm mục tiêu (Objective function)
26     obj_func = 3 * x + y
27     solver.Maximize(obj_func)
28
29     # Giải bài toán (Solve the problem)
30     status = solver.Solve()
31
32     return status, solver.Objective().Value(), x.solution_value(), y.solution_value()
33
34
35 def main():
36     solve_model()
37     status, obj_val, x_val, y_val = solve_model()
38     if status == pywraplp.Solver.OPTIMAL:
39         print("Solution:")
40         print("Objective value =", obj_val)
41         print("x =", x_val)
42         print("y =", y_val)
43
```

Solution:
Objective value = 4.0
x = 1.0
y = 1.0

Google OR-Tools: example 2

- Tìm k số nguyên nhỏ nhất trong list $X = [5, 3, 4, 2, 7]$
- Biến quyết định (decision variables)
 - ...
- Các ràng buộc (constraints)
 - ...
- Hàm mục tiêu (objective function)
 - ...

Google OR-Tools: example 2



```
6 from ortools.linear_solver import pywraplp
7
8 def solve_model(X, k):
9     # Tạo linear solver với SCIP (Solving Constraint Integer Programs)
10    solver = pywraplp.Solver.CreateSolver("SCIP")
11
12    n = len(X)
13    # Các biến quyết định (Decision variables)
14    y = [solver.IntVar(0, 1, f"y[{i}]") for i in range(n)]
15
16    # Các ràng buộc (Constraints)
17    solver.Add(solver.Sum(y) == k)
18
19    # Hàm mục tiêu (Objective function)
20    solver.Minimize(solver.Sum(X[i] * y[i] for i in range(n)))
21
22    # Giải bài toán (Solve the problem)
23    status = solver.Solve()
24
25    return status, solver.Objective().Value(), \
26           [y[i].solution_value() for i in range(n)]
27
```

Google OR-Tools: example 2

```
29 def main():
30     X = [5, 3, 4, 2, 7]
31     k = 1
32     status, obj_val, y = solve_model(X, k)
33     if status == pywraplp.Solver.OPTIMAL:
34         print("Solution:")
35         print("Objective value =", obj_val)
36         print("y =", y)
37
```

```
>>> %Run ortools_example_3.py
Solution:
Objective value = 2.0
y = [0.0, 0.0, 0.0, 1.0, 0.0]
>>>
```

Số nhỏ nhất ở vị trí
(index) thứ 3

Google OR-Tools: example 2

Examples: Amphibian Coexistence

- Trong bể nuôi có 3 loài sinh vật (mỗi loài một con) là toad, salamander, caecilian), các mồi ăn của chúng là 3 loại khác nhau: worms, crickets, and flies.
 - Mỗi ngày cho vào bể 1.500 worms, 3.000 crickets, và 5.000 flies. Mỗi loài sinh vật sẽ tiêu thụ một số lượng nhất định các con mồi trong ngày.
 - Có bao nhiêu loài sinh vật (tối đa 1.000 con mỗi sinh vật) có thể cùng tồn tại trong bể với số con mồi cung cấp mỗi ngày.

Table 1-2. Number of Preys Consumed by Each Species of Amphibian

Food	Toad	Salamander	Caecilian	Available
Worms	2	1	1	1500
Crickets	1	3	2	3000
Flies	1	2	3	5000

Examples: Amphibian Coexistence

1. Xác định vấn đề cần giải quyết (Identify the question to answer)
 - Bao nhiêu sinh vật mỗi loài cùng tồn tại trong bể?
 - Thiết lập câu hỏi chính xác qua các biến quyết định (decision variables: gọi x_0 , x_1 , x_2 là số lượng các sinh vật cùng tồn tại trong bể.
2. Xác định tất cả các yêu cầu và biểu diễn thành các ràng buộc [toán học] (Identify all requirements and translate them into constraints)
 - $2x_0 + x_1 + x_2 \leq 1500$
 - $x_0 + 3x_1 + 2x_2 \leq 3000$
 - $x_0 + 2x_1 + 3x_2 \leq 5000$
3. Xác định mục tiêu cần tối ưu (Identify the objective to optimize)
 - Hàm mục tiêu (objective function): $\max(x_0 + x_1 + x_2)$

Examples: Amphibian Coexistence

- Giải quyết bài toán tối ưu trên máy tính như thế nào?
 - Chuyển các biểu thức toán học thành dạng mà các chương trình giải (solvers) có thể hiểu được và thực thi
- Các ngôn ngữ mô hình hóa (modeling languages)
 - Các cấu trúc hình thức với các từ vựng và cú pháp đặc thù (specific vocabulary & grammars)
- Các chương trình giải (solvers)
 - Các gói phần mềm có thể đọc các mô hình (models) được biểu diễn bằng ngôn ngữ nhất định và đưa ra các nghiệm/lời giải (solutions).

Examples: Amphibian Coexistence

■ Các ngôn ngữ mô hình hóa (modeling languages)

- AMPL (www.ampl.com)
- GAMS (www.gams.com)
- GMPL (<http://en.wikibooks.org/wiki/GLPK/>)
- GMPL (MathProg))
- Minizinc (www.minizinc.org/)
- OPL ([www-01.ibm.com/software/info/ilog/](http://01.ibm.com/software/info/ilog/))
- ZIMPL (<http://zimpl.zib.de/>)

■ Các trình giải (solvers)

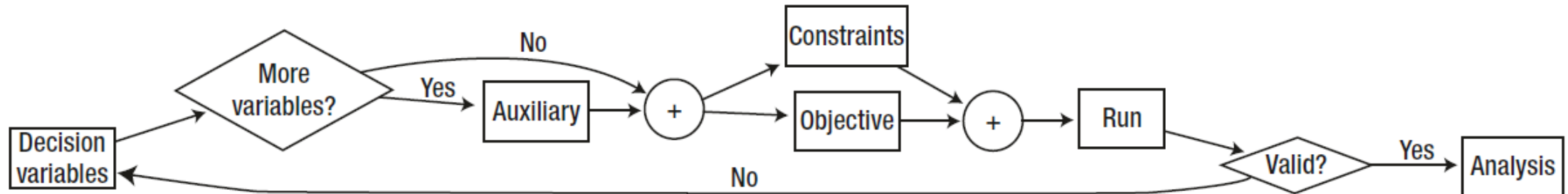
- CBC (www.coin-or.org/)
- CLP (www.coin-or.org/Clp/)
- CPLEX (www-01.ibm.com/software/info/ilog/)
- ECLiPSe (<http://eclipseclp.org/>)
- Gecode (www.gecode.org/)
- GLOP (<https://developers.google.com/optimization/lp/glop>)
- GLPK (www.gnu.org/software/glpk/)
- Gurobi (www.gurobi.com/)
- SCIP (<http://scip.zib.de/>)

Basic steps for solving an LP problem

- Các bước cơ bản để giải một bài toán Quy hoạch tuyến tính (Linear programming – LP)
 1. Khai báo thư viện (Import the linear solver wrapper)
 2. Khai báo trình giải (Declare the LP solver)
 3. Xác định các biến (Define the variables)
 4. Thiết lập các ràng buộc (Define the constraints)
 5. Định nghĩa hàm mục tiêu (Define the objective)
 6. Gọi solver thực thi (Call the LP solver)
 7. Hiển thị kết quả/nghiệm (Display the solution)

The steps to construct and run a model

■ Các bước để tạo và chạy một mô hình



GLOP (Google Linear Optimization Package)

- GLOP là “*linear programming solve*” trong Google OR-Tools
 - Được thiết kế để giải các bài toán tối ưu tuyến tính lớn (large-scale linear optimization problems) một cách hiệu quả.

Code 1

```
1 # Khai báo thư viện
2 from ortools.linear_solver import pywraplp
3
4 def solve_coexistence():
5     title_name = 'Amphibian coexistence'
6     # khai báo/tạo solver
7     solver = pywraplp.Solver(title_name, pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
8
9     # Các biến quyết định (Decision variables)
10    # x: mảng 1 chiều chứa 3 biến quyết định x0, x1, x2 trong khoảng [0, 1000])
11    x = [solver.NumVar(0, 1000, 'x[%i]' % i) for i in range(3)]
12
13    # Định nghĩa các ràng buộc (Constraints)
14    # (có thể viết thành các biểu thức toán học với nhiều cách khác nhau)
15    # ví dụ  $a - b \geq 0$  hay  $a \geq b$  đều được
16    solver.Add(2 * x[0] + x[1] + x[2] <= 1500)
17    solver.Add(x[0] + 3 * x[1] + 2 * x[2] <= 3000)
18    solver.Add(x[0] + 2 * x[1] + 3 * x[2] <= 4000)
```

Code 1

```
20 # Định nghĩa hàm mục tiêu (Objective function)
21 solver.Maximize(x[0] + x[1] + x[2])
22
23 # Gọi solver để giải bài toán
24 solver.Solve()
25
26 # Trả về giá trị tối ưu của hàm mục tiêu và các biến quyết định
27 return [round(entry.SolutionValue()) for entry in x]
28
29
30 def main():
31     x = solve_coexistence()
32     print('Species', x)
33
```

```
>>> %Run coexistence.py
Species [100, 300, 1000]
```

Code 2

```
1 # Import the linear solver wrapper
2 from ortools.linear_solver import pywraplp
3
4
5 def solve_coexistence():
6     title_name = 'Amphibian coexistence'
7     # Declare/create the LP (Linear Programming) solver
8     solver = pywraplp.Solver(title_name, pywraplp.Solver.GLOP_LINEAR_PROGRAMMING)
9
10    # Create the decision variables in the range [0, 1000]
11    # (mảng 1 chiều x chứa 3 biến quyết định x0, x1, x2 trong khoảng [0, 1000])
12    x = [solver.NumVar(0, 1000, 'x[%i]' % i) for i in range(3)]
13
14    # Create the auxiliary variable to maximize
15    # (biến phụ lưu giá trị của hàm mục tiêu)
16    max_species = solver.NumVar(0, 3000, 'max species of amphibians')
17
```


Code 2

```
18 # Define the constraints
19 # (có thể viết thành các biểu thức toán học với nhiều cách khác nhau)
20 # ví dụ  $a - b \geq 0$  hay  $a \geq b$  đều được
21 solver.Add(2 * x[0] + x[1] + x[2] <= 1500)
22 solver.Add(x[0] + 3 * x[1] + 2 * x[2] <= 3000)
23 solver.Add(x[0] + 2 * x[1] + 3 * x[2] <= 4000)
24
25 # Define the total number of species (biến phụ chứa biểu thức của hàm mục tiêu)
26 solver.Add(max_species == x[0] + x[1] + x[2])
27
28 # Define the objective function
29 # Có thể viết trực tiếp solver.Maximize(x[0] + x[1] + x[2])
30 solver.Maximize(max_species)
31
32 # Invoke the solver
33 solver.Solve()
34
35 # Trả về giá trị tối ưu của hàm mục tiêu và các biến quyết định
36 return max_species.SolutionValue(), [round(e.SolutionValue()) for e in x]
```

Code 2

```
37
38
39 def main():
40     m, x = solve_coexistence()
41     print('Max species (count):', m)
42     print('Species', x)
43
44
45 if __name__ == '__main__':
46     main()
```

```
>>> %Run coexistence_2.py
Max species (count): 1400.0
Species [100, 300, 1000]
```

Read more...

- Get Started with OR-Tools for Python
 - <https://developers.google.com/optimization/introduction/python>

